



OFFICIAL TECHNICAL HANDOFF MANUAL

B² CLUB

RUN • BURN • BOND • MADURAI

B² Club Platform Transfer, Architecture & Hosting Manual

The comprehensive technical reference, infrastructure specification, ownership transfer runbook, and step-by-step hosting guide for the B² Club web application. Written so that any developer, agency, or technical lead can pick up, transfer, deploy, and maintain this system with zero ambiguity.

PRODUCTION WEB APPLICATION

<https://b2club.in> (b2club.in)

SOURCE CODE REPOSITORY

github.com/Nigazhvan-31/b2runclub (branch: main)

PAYMENT GATEWAY

Razorpay Payment Gateway (Live Merchant Mode)

BACKEND SERVERLESS API

<https://runclub-backend-navy.vercel.app>

DATABASE & OBJECT STORAGE

Postgres (Supabase Managed) & Vercel Blob

PRIMARY ADMIN CONTACT

burnandbond.club@gmail.com

TABLE OF CONTENTS

§1. Executive Summary & Application Overview	Page 3
§2. Complete Ownership Transfer Playbook (Step-by-Step Handover)	Page 4
§3. Architecture, Hosting Topology & Tech Stack	Page 6
§4. Environment Variables Reference (Exhaustive Table)	Page 8
§5. Database Architecture & Prisma Migration Engine	Page 11
§6. Feature-by-Feature Implementation & Business Rules	Page 13
§7. API Reference Specification	Page 15
§8. Scheduled Jobs, Automation & Webhooks	Page 17
§9. Payment Processing & Mobile App Eviction Recovery	Page 18
§10. Media Pipeline & "Any Pixel Any Size" Optimizer	Page 19
§11. Zero-to-Live Production Hosting Guide	Page 20
§12. Local Development Setup (SQLite) & Testing	Page 22
§13. Emergency Runbook & Troubleshooting Guide	Page 23

§1. Executive Summary & Application Overview

B² Club is an all-in-one membership management, event ticketing, race-day execution, and community platform built for Madurai's premier running club. The application enables runners to discover upcoming weekend runs, treks, badminton leagues, and socials; sign up as individuals or parties; secure payments online via Razorpay UPI and cards; hold spots under an automated 24-hour reservation window; and receive verifiable QR code tickets scanned on the morning of each event.

For organizers, the platform provides a complete Race-Day Console: participant roster management with phone numbers for WhatsApp group creation, person-by-person check-in scanning, marshal shift volunteer tracking, split-timing checkpoints, Excel spreadsheet exports, and community photo galleries.

1.1 User Roles & Permissions Matrix

The application defines four distinct user roles stored as plain uppercase strings on `User.role` :

Role	Permissions & Capabilities	Access Gating	Pricing Policy
VISITOR	Browse published events, view community photo gallery, read forum announcements, inspect club founders, view polls.	Unauthenticated or logged-out users default to this role.	N/A (cannot register without an account).
MEMBER	All visitor privileges, plus: register for events (adults & kids), hold unpaid tickets for 24h, receive QR tickets, post comments in the forum, vote in polls, upload photos to the gallery, and track workout attendance.	Requires a confirmed email address (one-time OTP verification).	Standard adult & kid event entry fees apply. Eligible for party group discounts.
VOLUNTEER	All member privileges, plus: can claim marshal shifts on race morning, run the mobile QR camera ticket scanner to admit participants, and manage start-line check-ins.	Assigned by an Administrator from the members directory.	Free entry for the volunteer's own spot. Any additional guests they register pay standard fees.
ADMIN	Full platform authority: create, edit, draft, and publish events; view financial revenue overview; manage all registrations; issue Razorpay refunds; export roster workbooks (.xlsx); trigger hold/reminder sweeps; promote member roles; manage founders.	Authenticated via JWT with role <code>ADMIN</code> .	Full administrative access across all screens. Automatically routes to race-day console.

§2. Complete Ownership Transfer Playbook

When handing over this application to a new owner, technical agency, or client IT team, six external services must be transferred in sequence. Follow this exact playbook to guarantee zero downtime and no loss of active runner data or payments.

⚠ Handover Principle: Keep Services Active in Parallel

Never shut down existing Vercel deployments or revoke existing database credentials until the recipient has verified access on their own accounts. Transfer organizational ownership rather than recreating projects wherever possible.

2.1 Git Repository Transfer (GitHub)

1. **Transfer Repository:** Navigate to `GitHub → Settings → General → Danger Zone → Transfer ownership`. Enter the recipient's GitHub username or Organization handle (e.g. `b2runclub-org`).

2. **Update Remote URLs:** The new developer runs:

```
git remote set-url origin https://github.com/NEW_OWNER/b2runclub.git
git fetch --all
```

3. **Maintain Branch Protection:** Ensure branch protection on `main` requires passing status checks before merging.

2.2 Vercel Project & Custom Domain Transfer

The application runs as two distinct Vercel projects: **Frontend** (SPA) and **Backend** (Serverless Function).

1. **Invite Recipient:** In Vercel, go to your Team/Account settings → `Members` → Invite the client email with *Admin* or *Owner* privileges.

2. **Transfer Projects:**

- Frontend Project: Go to `Settings → General → Transfer Project` → Transfer to recipient's team.
- Backend Project: Repeat for the backend serverless project.

3. **Custom Domain Verification:**

- Domain `b2club.in` and `www.b2club.in` must remain assigned to the **Frontend project**.
- DNS records at the registrar (e.g. Hostinger / GoDaddy):
 - **A Record**: `@` points to `76.76.21.21`
 - **CNAME Record**: `www` points to `cname.vercel-dns.com`

2.3 Supabase Database & Storage Transfer

1. **Invite to Organization:** In the Supabase Dashboard, select your Organization → `Members` → Invite new owner with *Owner* role.

2. **Transfer Ownership:** Once accepted, transfer primary organizational billing and ownership.

3. **Rotate Database Passwords:** Go to `Project Settings → Database → Database password`. If resetting the password, immediately update `DATABASE_URL` and `DIRECT_URL` in Vercel backend environment variables!
4. **Verify Storage Bucket:** Confirm that the storage bucket `uploads` is set to *Public* so that uploaded photos, avatars, and route maps remain accessible.

2.4 Razorpay Payment Gateway Handover

1. **Change Merchant Account Owner:** In Razorpay Dashboard → `Settings → Account Settings → Users`. Invite the new business administrator and transfer the primary account owner role.
2. **Bank Account Verification:** Verify that the linked bank account for daily settlements is updated to the club's current commercial bank account.
3. **API Keys & Webhooks Handover:**
 - Provide the active `RAZORPAY_KEY_ID` and `RAZORPAY_KEY_SECRET` (in *Live* mode).
 - Verify the Webhook URL: `https://runclub-backend-navy.vercel.app/api/payments/webhook` with secret `RAZORPAY_WEBHOOK_SECRET`.

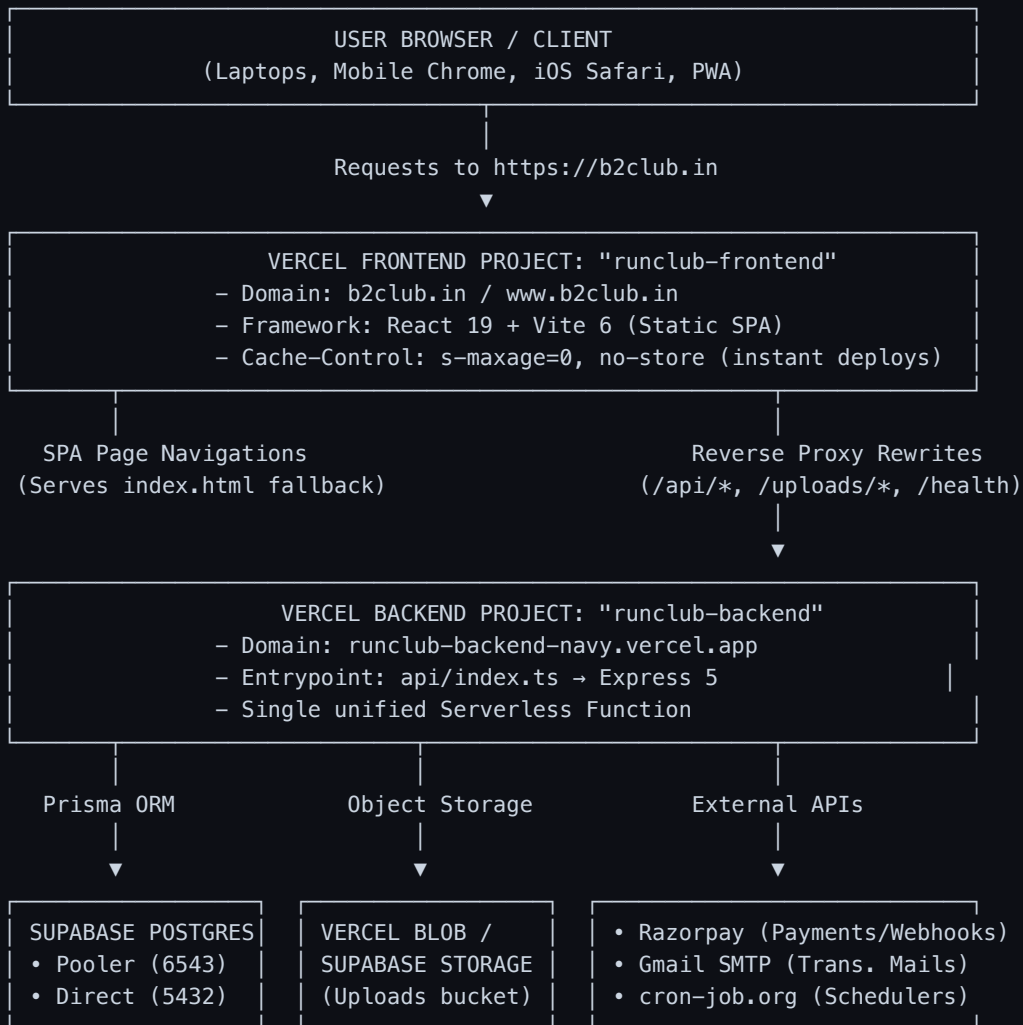
2.5 Email & Google Workspace Handover

1. **Email Account:** Primary outgoing mail is sent via `burnandbond.club@gmail.com`.
2. **Google App Password:** If the Gmail password is changed, Google automatically revokes all active App Passwords. The new owner must immediately visit myaccount.google.com/apppasswords, generate a new 16-character App Password, and save it to `SMTP_PASS` in Vercel.

CLUB
— RUN • BURN • BOND —

§3. Architecture, Hosting Topology & Tech Stack

The platform is architected as two serverless services managed on Vercel, entirely eliminating server maintenance, OS patching, and capacity provisioning.



3.1 Why the Frontend Proxies the API (No CORS)

The frontend's `vercel.json` rewrites all requests matching `/api/:path*`, `/uploads/:path*`, and `/health` directly to `https://runclub-backend-navy.vercel.app`.

- **Zero Cross-Origin Hassle:** Browsers communicate strictly with `b2club.in`. Cookies, authorization headers, and multipart form uploads behave as single-origin requests.
- **Client SPA Fallback:** `/(.*) -> /index.html` ensures deep links like `/events/abc` or `/raceday/123` load cleanly on hard browser refresh without 404s.
- **Backend Decoupling:** If the backend URL ever changes, update only the 3 rewrite lines in `runclub-frontend/vercel.json`. No frontend code re-compilation is required.

3.2 Technical Stack Specifications

Layer	Technologies & Libraries	Version / Role
Frontend Core	React 19, TypeScript, Vite 6, Tailwind CSS 4	Modern, zero-latency reactive SPA.
UI & Animation	Framer Motion, GSAP, three.js / react-three-fiber	Physics-based cards, tilt effects, 3D scenes.
Barcode / QR	<code>jsqr</code> (camera barcode reader), <code>qrcode</code> (generation)	High-speed start-line participant check-in.
Backend Runtime	Node.js 20+, Express 5, TypeScript	Vercel Serverless Function engine.
ORM & Database	Prisma 7, <code>@prisma/adapters-pg</code> , PostgreSQL (Supabase)	Connection pooled, type-safe queries.
Authentication	JWT (<code>jsonwebtoken</code>), bcrypt/crypto password hashing	Stateless bearer tokens, 24-hour expiration.
Spreadsheets	ExcelJS	Server-side real <code>.xlsx</code> roster generation.
Payments	Razorpay SDK (Node), Razorpay Checkout.js (Browser)	UPI, Cards, NetBanking, auto-reconciliation.



§4. Environment Variables Reference (Exhaustive)

Configure these variables in **Vercel → Project Settings → Environment Variables**. For local development, save them in `runclub-backend/.env`.

4.1 Backend Environment Variables

Variable Name	Purpose & Description	Format / Critical Notes
<code>DATABASE_URL</code>	Postgres connection URI for application queries.	Must use Supabase Pooler (Port 6543) with <code>?pgbouncer=true</code> . Prevents exhausting database connection limits in serverless environments.
<code>DIRECT_URL</code>	Direct Postgres connection URI used exclusively for migrations.	Must use Direct Port 5432 . The pooler's transaction mode cannot execute prepared statements needed by <code>prisma migrate</code> .
<code>JWT_SECRET</code>	Cryptographic secret used to sign and verify user authentication tokens.	Must be a high-entropy string of at least 32 characters. Rotating this logs all active users out.
<code>APP_URL</code>	Public frontend URL used to format links in transactional emails.	Set to <code>https://b2club.in</code> . A missing or incorrect value generates broken links in verification and password reset emails.
<code>RAZORPAY_KEY_ID</code>	Publishable Merchant Key ID from Razorpay.	Format: <code>rzp_live_...</code> in production, <code>rzp_test_...</code> for local testing. Publicly exposed to client browser via <code>/api/payments/config</code> .
<code>RAZORPAY_KEY_SECRET</code>	Secret key used to verify payment signatures and issue refunds.	Private secret. Never expose to client bundles.
<code>RAZORPAY_WEBHOOK_SECRET</code>	Secret key used to verify HMAC-SHA256 signatures of incoming payment webhooks.	Must match the secret entered in the Razorpay Webhooks dashboard. If unset, webhooks fail with 503.
<code>SMTP_HOST</code>	Host for outgoing transactional email server.	Set to <code>smtp.gmail.com</code> .
<code>SMTP_PORT</code>	Port for SMTP server.	Set to <code>587</code> (STARTTLS) or <code>465</code> (SSL/TLS).
<code>SMTP_USER</code>	SMTP login email address.	Set to <code>burnandbond.club@gmail.com</code> .
<code>SMTP_PASS</code>	SMTP password.	Must be a 16-character Google App Password , not the regular Gmail account password.

Variable Name	Purpose & Description	Format / Critical Notes
<code>MAIL_FROM</code>	Sender header on outgoing emails.	Format: <code>"B² Club"</code> <code><burnandbond.club@gmail.com></code> .
<code>CRON_SECRET</code>	Authorization token protecting background maintenance endpoints.	Protects <code>/api/cron/reminders</code> and <code>/api/cron/holds</code> from public invocation.
<code>ADMIN_EMAIL</code>	Email address for automated administrative account seeding.	Used by <code>dist/seed.js</code> on fresh database provisioning.
<code>ADMIN_PASSWORD</code>	Password for initial administrative account seeding.	Minimum 8 characters. Exits early if an admin already exists.
<code>BLOB_READ_WRITE_TOKEN</code>	Vercel Blob token for cloud photo storage.	Auto-injected by Vercel when attaching a Blob store to the backend project.

4.2 Frontend Environment Variables

Variable Name	Recommended Value	Reasoning
<code>VITE_API_BASE_URL</code>	<i>(Leave completely empty)</i>	Leaving empty instructs the frontend to call relative endpoints (e.g. <code>/api/...</code>), cleanly handled by Vercel rewrites without CORS.
<code>VITE_RAZORPAY_KEY_ID</code>	<code>rzp_live_...</code> (Optional fallback)	Fallback only. The frontend dynamically fetches the active key ID from <code>GET /api/payments/config</code> .

— RUN • BURN • BOND —

§5. Database Architecture & Migration Engine

The persistent data layer is hosted on Supabase PostgreSQL and managed via Prisma ORM.

5.1 The Two-Port Rule: Connection Pooler vs. Direct Port

⚠ Critical Database Connection Architecture

Supabase exposes two connection ports. Conflating them will cause either application outages or migration crashes:

- **Port 6543 (Pooler):** Used by `DATABASE_URL`. Employs PgBouncer in transaction mode to multiplex connections from serverless functions. *Cannot run prepared statements.*
- **Port 5432 (Direct):** Used by `DIRECT_URL`. Bypasses the pooler to establish a stateful session required by `prisma migrate`.

5.2 Core Data Models & Relationships

- `User`: Stores authentication credentials, full name, role (`MEMBER`, `VOLUNTEER`, `ADMIN`), emergency contact, and mandatory normalized E.164 phone number.
- `Event`: Defines club sessions (e.g. `WEEK 27 - B2 TREKKING`), discipline, UTC timestamp, venue, capacity, kid pricing rules, party discount rules, spot hold duration, cover image, and GPX route path.
- `EventRegistration`: Represents one booking order. Tracks Razorpay order ID, payment status, pricing snapshots, `hold_expires_at` deadline, and cancellation timestamps.
- `RegistrationGuest`: **One record per participant**. Captures attendee name, classification (`ADULT` or `KID`), whether they are the primary booker, their own WhatsApp phone number, and physical race-morning admission timestamp (`admitted_at`).
- `Photo`: Community gallery images tagged to events, tracking uploader reference, caption, and object storage URL.
- `EventShift` / `ShiftAssignment`: Marshal and volunteer shift rosters for race day.

5.3 Registration Status State Machine

Stored Status	Public Display Label	Reserves Spot?	Ticket QR Accessible?
PENDING	Awaiting Payment	Yes (until <code>hold_expires_at</code>)	No (must be completed first)
PAID	Paid	Yes	Yes (active verifiable QR ticket)
FREE	Free (Volunteer Entry)	Yes	Yes (active verifiable QR ticket)
EXPIRED	Spot Expired	No (released to public)	No
CANCELLED	Cancelled	No	No

Stored Status	Public Display Label	Reserves Spot?	Ticket QR Accessible?
DEACTIVATED	Deactivated (Admin action)	No	No

5.4 The Automated Pre-Migrate Reconciliation Script

Located at `runclub-backend/scripts/pre-migrate.mjs`, this script runs automatically before every `npx prisma migrate deploy` in the Vercel build command:

- Removes any lingering failed migrations (where `finished_at IS NULL`) preventing Prisma error `P3009`.
- Normalizes migration prefixes to two-digit zero-padded folder conventions (e.g. `00_init`).



§6. Feature-by-Feature Implementation Guide

6.1 Party Bookings & Phone Number Collection

A single member can register up to **6 attendees** in a single booking:

- **Individual Mobile Numbers:** For every guest in the party, their personal WhatsApp number is gathered so organizers can compile event coordination WhatsApp groups.
- **"Use My Number" Flag:** For children or spouses without a personal phone, a checkbox copies the booker's number. Resolved securely on the backend to prevent payload tampering.
- **Emergency Contact Validation:** The booker must supply a distinct emergency phone number. A safety validation rule rejects submissions where the emergency contact matches the runner's own phone.

6.2 The 24-Hour Spot Hold & Read-Time Expiry Engine

When a runner begins booking an event with limited capacity, the system creates a `PENDING` registration and computes a hold deadline (`hold_expires_at = NOW() + hold_minutes`, default 24 hours).

💡 Read-Time Availability Filter (Zero Delay Release)

Rather than waiting for a background cron job to mark expired bookings, `seatFilter()` in `events.router.ts` evaluates hold expiration **at read time**. The moment an unpaid reservation crosses its expiration timestamp, the capacity counter immediately releases the spot back to other runners.

6.3 QR Ticketing & Start-Line Check-In

- **Party-Wide QR Code:** One encrypted QR code covers the entire registration group.
- **Camera-Based Scanner:** Marshals open `/scan` on their mobile browsers, utilizing `jsqr` to scan tickets directly via device cameras.
- **Person-by-Person Admission:** Marshals can mark individual attendees in a family as checked-in as they arrive at the start line. Check-in actions can be undone if clicked in error.

6.4 Community Photo Gallery & "Any Pixel Any Size" Optimizer

The community gallery at `/gallery` allows runners and organizers to submit photos from club sessions:

- **Universal Permissions:** Authenticated `MEMBER`, `VOLUNTEER`, and `ADMIN` accounts can post photos. Non-admins can delete only their own uploaded photos.
- **Client-Side 2.5K Canvas Optimizer:** Handled by `optimiseImageFile` in `Gallery.tsx`. Large camera files up to 25MB are scaled down in the browser to 2560px on their longest edge and compressed to ~500KB JPEG before upload. This guarantees uploads never hit Vercel's 4.5MB serverless payload limit.
- **Adaptive Masonry & Lightbox:** Pure CSS columns and flexbox ensure ultra-wide landscape, tall portrait, and square photos render with their natural aspect ratio without cropping or stretching.

6.5 Server-Generated Excel Roster Exports (.xlsx)

The application exports native Excel workbooks via `ExcelJS` (no CSV format drift):

- **Event Roster:** One row per participant including attendee category, ticket status, payment reference, emergency contact, and questionnaire answers.
- **All Rosters:** Master export of all historical event entries.
- **Member Directory:** Complete runner database with attendance streaks and payment histories.



§7. API Reference Specification

All endpoints are mounted under `/api`. Global health check is available at `GET /health`.

7.1 Authentication Endpoints (`/api/auth`)

Method	Endpoint	Access	Description
POST	<code>/register</code>	Public	Registers a new member (validates E.164 phone, hashes password).
POST	<code>/login</code>	Public	Returns JWT session token (24h validity) and user profile.
POST	<code>/forgot-password</code>	Public	Issues cryptographic 45-minute password reset link via email.
POST	<code>/reset-password</code>	Public	Applies new password using valid single-use token.
GET	<code>/me</code>	Signed In	Returns current authoritative account profile and role from DB.
PATCH	<code>/me</code>	Signed In	Updates user name, emergency contact, or phone number.

7.2 Event & Booking Endpoints (`/api/events`)

Method	Endpoint	Access	Description
GET	<code>/</code>	Public	Lists upcoming and active published club events.
GET	<code>/:id</code>	Public	Returns event details, capacity counters, pricing, and questionnaire.
POST	<code>/</code>	ADMIN	Creates a new draft or published event.
PUT	<code>/:id</code>	ADMIN	Updates event settings, pricing, cover image, or reminders.
DELETE	<code>/:id</code>	ADMIN	Soft-deletes or removes an unpublished event.
POST	<code>/:id/register</code>	Verified Member	Creates registration, initiates 24h hold, and creates Razorpay order.
GET	<code>/me/registrations</code>	Signed In	Returns booking history and QR tickets for current user.
GET	<code>/registration/:id/ticket</code>	Owner / ADMIN	Renders standalone printable/scannable HTML QR ticket.

7.3 Payment & Reconciliation Endpoints (/api/payments)

Method	Endpoint	Access	Description
GET	/config	Public	Provides publishable Razorpay Key ID and mock-mode status.
POST	/verify	Signed In	Verifies browser payment signature and transitions registration to PAID.
POST	/webhook	Razorpay	Signature-verified webhook capturing <code>payment.captured</code> events.
POST	/reconcile/:id	Owner / ADMIN	Queries Razorpay API directly to settle an unconfirmed booking.
POST	/refund/:id	ADMIN	Issues automated refund through Razorpay bounded by amount paid.

7.4 Gallery & Content Endpoints (/api/content)

Method	Endpoint	Access	Description
GET	/gallery	Public	Fetches community photos (optionally filtered by <code>?event_id=</code>).
POST	/gallery	Member / Admin	Uploads a photo (multipart/form-data) with caption and event tag.
DELETE	/gallery/:id	Member / Admin	Removes photo (admins can delete any; members delete own).

— RUN • BURN • BOND —

§8. Scheduled Jobs, Background Sweeps & Webhooks

8.1 The Two Automated Sweeps

Endpoint	Frequency	Operational Purpose
<code>/api/cron/reminders</code>	Daily (01:30 UTC / 07:00 IST)	Scans upcoming events and dispatches scheduled email reminders based on configured hour offsets (e.g. 24h, 48h before start). Uses a 6-hour grace window to prevent stale sends.
<code>/api/cron/holds</code>	Daily (02:00 UTC / 07:30 IST)	Audits unpaid <code>PENDING</code> bookings past their <code>hold_expires_at</code> timestamp. Updates database rows to <code>EXPIRED</code> and dispatches friendly reminder nudges.

8.2 The Vercel Hobby Plan Limitation & External Scheduler Fix

⚠ Vercel Hobby Cron Limitation

Vercel Hobby accounts permit cron jobs to run **only once per day**. If you configure a "2 hours before start" event reminder, a once-daily cron may miss it.

The Production Solution: Set up a free external cron monitor (such as cron-job.org or [UptimeRobot](https://uptimerobot.com)) to ping both endpoints every 10–15 minutes:

```
https://runclub-backend-navy.vercel.app/api/cron/reminders?key=YOUR_CRON_SECRET
https://runclub-backend-navy.vercel.app/api/cron/holds?key=YOUR_CRON_SECRET
```

§9. Payment Processing & Mobile App Eviction Recovery

9.1 The Mobile Browser Tab Eviction Problem

In India, over 80% of runners pay via mobile UPI apps (Google Pay, PhonePe, Paytm). During checkout, the mobile browser transfers focus to the UPI app. On budget or memory-constrained mobile devices, the Android OS frequently kills the background browser tab to free RAM.

When the user completes payment in GPay and returns, the browser reloads from scratch. The client-side callback never runs, leaving the booking stuck in `PENDING` even though money was deducted from the runner's bank account.

9.2 The Dual Defense Architecture

1. **Razorpay Server-to-Server Webhook:** Razorpay notifies `POST /api/payments/webhook` directly from server to server. The endpoint verifies the signature using `RAZORPAY_WEBHOOK_SECRET` and marks the booking `PAID` regardless of what happened to the user's browser tab.
2. **Pre-Payment Reconcile Check:** If a runner returns to the site and clicks "Pay Now" on a stuck booking, the application calls `POST /api/payments/reconcile/:id` before initiating a new Razorpay checkout. The server queries the Razorpay API, detects the existing capture, settles the ticket, and immediately displays the QR code without charging the runner a second time.

CLUB
— RUN • BURN • BOND —

§10. Media Pipeline & "Any Pixel Any Size" Optimizer

The photo upload system is designed to handle photography directly from modern high-end smartphones without crashing serverless functions or distorting the layout.

10.1 The Vercel 4.5MB Serverless Ceiling

Vercel serverless functions enforce a strict incoming request body size limit of **4.5 MB**. Photos taken directly with modern iPhone or Android cameras (12MP, 48MP, 108MP) typically range from **5 MB to 20 MB**. Posting these files directly to the serverless backend causes Vercel's edge proxy to terminate the connection with HTTP `413 Payload Too Large` or `403 Forbidden`.

10.2 The Solution: Client-Side 2.5K Canvas Resizing

Implemented in `runclub-frontend/src/pages/Gallery.tsx` via `optimiseImageFile()`:

- The file input accepts raw image files up to **25 MB** (JPEG, PNG, WebP, GIF, AVIF).
- Before dispatching the network request, the image is loaded into an HTML5 `<canvas>` element.
- If either dimension exceeds **2560px** (2.5K resolution), the longest edge is scaled to 2560px with aspect ratio strictly preserved.
- The canvas exports a compressed JPEG at `0.88` quality, reducing the file size to **~400 KB – 800 KB** with zero visible quality loss.
- The upload completes in under 1 second, completely bypassing Vercel payload restrictions.

— RUN • BURN • BOND —

§11. Zero-to-Live Production Hosting Guide

Follow these steps to deploy a completely fresh instance of B² Club from scratch on new hosting accounts.

Step 1: Set Up Supabase Database & Storage

1. Create a new project on supabase.com in the Mumbai region (`ap-south-1`).
2. Navigate to `Project Settings → Database` :
 - Copy the **Transaction Pooler URI** (Port `6543`) → save as `DATABASE_URL` .
 - Copy the **Direct Connection URI** (Port `5432`) → save as `DIRECT_URL` .
3. Navigate to `Storage` → Create a new bucket named `uploads` → Toggle **Public Bucket** to ON.

Step 2: Deploy Backend to Vercel

1. In Vercel, import the GitHub repository `b2runcclub` .
2. Set **Project Name**: `runcclub-backend` .
3. Set **Root Directory**: `runcclub-backend` .
4. Set **Build Command**:

```
npm run build && node scripts/pre-migrate.mjs && npx prisma migrate deploy && node dist/seed
```
5. Add all environment variables listed in §4 (Database URIs, JWT Secret, Razorpay keys, Gmail SMTP credentials, Admin Seed).
6. Click **Deploy**. Copy the assigned backend URL (e.g. `https://runcclub-backend-navy.vercel.app`).

Step 3: Deploy Frontend to Vercel

1. In Vercel, import the same GitHub repository a second time.
2. Set **Project Name**: `runcclub-frontend` .
3. Set **Root Directory**: `runcclub-frontend` .
4. Framework Preset: `Vite` . Build Command: `npm run build` . Output Directory: `dist` .
5. Verify that `runcclub-frontend/vercel.json` rewrites point to your active backend URL from Step 2:

```
"rewrites": [  
  { "source": "/api/:path*", "destination": "https://YOUR-BACKEND.vercel.app/api/:path*" },  
  { "source": "/uploads/:path*", "destination": "https://YOUR-BACKEND.vercel.app/uploads/:path*" },  
  { "source": "/health", "destination": "https://YOUR-BACKEND.vercel.app/health" },  
  { "source": "/(.*)", "destination": "/index.html" }  
]
```

6. Click **Deploy**.

Step 4: Attach Custom Domains & DNS Records

1. In the Frontend Vercel Project → **Settings → Domains** : Add **b2club.in** and **www.b2club.in** .
2. In your DNS provider (e.g. Hostinger / GoDaddy):
 - **A Record** : **@** → **76.76.21.21**
 - **CNAME Record** : **www** → **cname.vercel-dns.com**
3. Vercel automatically provisions and renews SSL/TLS certificates via Let's Encrypt.

Step 5: Post-Deployment Smoke Test

- Visit **https://b2club.in/health** → confirms **{"status":"healthy","database":"connected"}** .
- Visit **https://b2club.in/api/payments/config** → verifies active Razorpay Key ID and mock mode false.
- Log in using seeded **ADMIN_EMAIL** and **ADMIN_PASSWORD** .
- Visit **/admin/mailer** → verify that SMTP shows connected with zero send errors.
- Upload a photo in **/gallery** → confirm that client-side 2.5K image optimization fits and publishes cleanly.



§12. Local Development Setup & Testing

12.1 Running Locally with SQLite

To ensure developers can work offline without paying for or configuring cloud Postgres instances, the codebase features an automated SQLite derivation engine:

```
# 1. Clone repository
git clone https://github.com/Nigazhvan-31/b2runclub.git
cd b2runclub

# 2. Start Backend (http://localhost:3000)
cd runclub-backend
npm install
npm run dev

# 3. Start Frontend in a second terminal (http://localhost:5173)
cd ../runclub-frontend
npm install
npm run dev
```

⚠ The Prisma Local Generation Rule

Running `npm run build` or `npx prisma generate` builds the client for production PostgreSQL. If you switch back to local SQLite development, run:

```
npm run generate:local
```

This executes `scripts/local-schema.mjs`, deriving a SQLite-compatible Prisma client for `dev.db`.

12.2 Automated API Test Suite

The repository includes an extensive end-to-end API regression test suite at `runclub-backend/scripts/api-test-suite.mjs`, executing 90 automated assertions covering registrations, holds, spot expirations, pricing rules, and role permissions:

```
cd runclub-backend
node scripts/api-test-suite.mjs
```

§13. Emergency Runbook & Troubleshooting Guide

13.1 Problem: Gallery Photo Upload Fails with 403 Forbidden

- **Cause:** Previous versions restricted photo uploads to Admins only, or the user's JWT carried a stale role.
- **Resolution:** In the latest build (commit `c42b479`), `POST /api/content/gallery` is open to all signed-in members (`MEMBER`, `VOLUNTEER`, `ADMIN`), and `requireRole` automatically revalidates against the live database if a token role is out of sync. Ensure users are logged into an active account before uploading.

13.2 Problem: Transactional Emails Stop Sending (Gmail SMTP 535)

- **Cause:** The Google account password was changed, which automatically invalidates all existing Google App Passwords.
- **Resolution:** Log into `burnandbond.club@gmail.com`, visit myaccount.google.com/apppasswords, generate a fresh 16-character App Password, update `SMTP_PASS` in Vercel backend environment variables, and redeploy.

13.3 Problem: Runner Paid Money via UPI but Ticket Stays "Awaiting Payment"

- **Cause:** Mobile browser tab eviction killed the frontend payment callback before it finished.
- **Resolution:**
 1. Advise runner to click "Pay Now" on their profile → the system runs `reconcile` and instantly releases the ticket without charging again.
 2. Or log into Admin Console → Manage Event Roster → Click **Reconcile Payments** to auto-settle all pending orders against Razorpay.